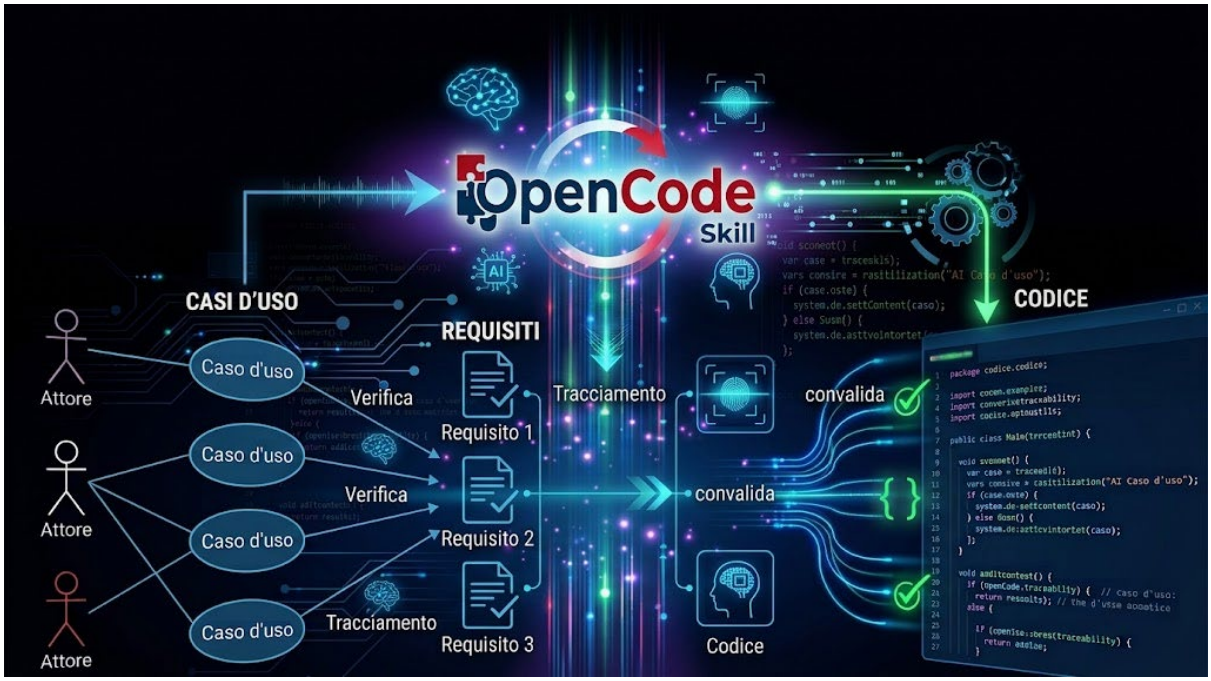
Università degli Studi di Padova
Corso di Ingegneria del Software 2026/2027



SpecCheck

Sviluppo di Skills per OpenCode per la verifica della tracciabilità end-to-end da casi d'uso a requisiti e codice sorgente.

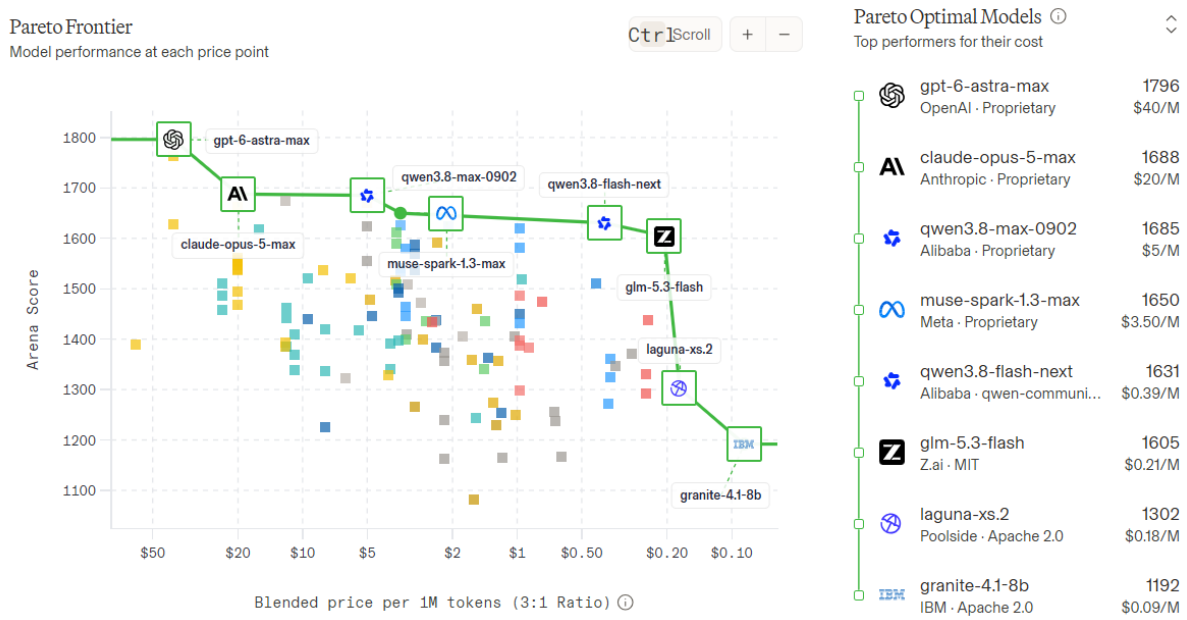
ZUCCHETTI
IL SOFTWARE CHE CREA SUCCESSO

Oggetto dell'appalto

I Large Language Models si stanno rivelando capaci di sviluppare codice in quasi tutti i linguaggi di programmazione.

Andrej Karpathy, figura di grande rilevanza per il settore, ha fatto la prima proposta per sfruttare questa possibilità: il "Vibe Coding". Lo sviluppatore trasmette le sue idee e le sue sensazioni riguardo al problema che vuole risolvere ad un LLM che realizza il codice.
Andrej Karpathy aveva ben caratterizzato questa idea con la frase "il prossimo linguaggio di programmazione sarà l'inglese".

La capacità di trattare il codice è stata affinata nel tempo portando a nuovi metodi di addestramento, nuove architetture, finestre di contesto sempre più grandi e nuove capacità di svolgere compiti di lunga durata composti di tante operazioni elementari.



Classifica degli LLM per il coding, frontiera di Pareto per costo

La crescita in questo settore è stata accompagnata dallo sviluppo di benchmark studiati per verificare le capacità effettive di questi sistemi. I test tradizionali, come MMLU (https://huggingface.co/datasets/cais/mmlu) e GSM8K (https://huggingface.co/datasets/openai/gsm8k) , infatti erano orientati più all'erudizione e al ragionamento che accompagna le attività di scrittura di testi.

I nuovi “SWE-Bench” (https://github.com/swe-bench/SWE-bench), “Terminal-bench” (https://www.tbench.ai/) e similari allargano l’attenzione alla pianificazione e allo svolgimento di attività correlate con lo sviluppo di applicazioni, dalla scrittura del codice all’esecuzione di operazioni di impacchettamento, dalla stesura di test alla scrittura della documentazione.

Sono bastati pochi esperimenti per comprendere che il “vibe coding” era una forma moderna di “cowboy coding” ed aveva le gambe molto corte, portando alla produzione di codice piagato da errori e parti mancanti.
Non bastava colloquiare con l’LLM attraverso il classico chatbot: la struttura del flusso di informazioni tra creatore ed esecutore è molto più ricca e le scelte architetturali e strutturali devono essere mantenute nel tempo, molto più a lungo dell'interazione “generativa” .

Sono così stati sviluppati i sistemi di gestione del contesto, che si occupano appunto di registrare il percorso creativo e mantenere le scelte di progetto.
Questi sistemi, come Claude Code, hanno arricchito l’LLM di informazioni che documentano e vincolano lo sviluppo del progetto. Diventano parti fondamentali i file come AGENTS.md che guidano il sistema mantenendo integre le scelte architetturali e l’insieme degli elementi come librerie e componenti durante tutto lo sviluppo.
Tutte queste attività operano su progetti di dimensione molto rilevante, molto di più dei testi su cui venivano usati gli LLM, conseguentemente non è sufficiente l’LLM con il ChatBot. Al suo posto sono stati sviluppati sistemi di gestione del contesto sempre più evoluti.
Oggi è consolidato che la gestione dell’ “harness”, cioè gestione del contesto ed impalcatura a sostegno delle attività di pianificazione e produzione del codice, con strumenti che creano riassunti, documenti di guida, elementi di riferimento per la comprensione del problema, hanno un ruolo che pesa quasi più di quello del modello LLM.
Tanto per dare una indicazione, sembra che i grandi risultati del benchmark SWE-bench di ChatGPT 6 Astra siano per un buon 30% dovuti al sistema di harness utilizzato.

RANK	METHOD	RESOLVED	NUM_VALID_INST	DATE
1	AMI Agent + Claude-4.6-Opus	63.0%	300	06/23/2026
2	agav-agent + GPT-5.5 (Medium) Verified	62.0%	300	09/03/2026
3	Slingshot-v3.3.0 + GPT-5.6-Sol Verified	44.7%	300	08/31/2026
4	SWE-agent + Claude-4.5-Sonnet	36.0%	300	12/01/2025
5	AiWork.Code + Claude-4.8-Opus (xhigh) Verified	34.0%	300	09/09/2026
6	Slingshot-v3.1.0 + Claude-4.6-Opus	25.0%	300	08/17/2026
7	OpenHands + Qwen3-Coder-480B-A35B	24.7%	300	07/25/2025

Classifica SWE-Bench di LLM+sistema di Harness a metà del 2026

I sistemi più diffusi per questo compito sono:

1. Claude Code, https://claude.com/it/product/claude-code
2. Deepseek harness, https://deepseek.com/harness
3. Hermes Agent, https://hermes-agent.nousresearch.com/
4. Codex, https://openai.com/it-IT/codex/
5. Antigravity, https://antigravity.google/
6. Kiro, https://kiro.dev/
7. Goose, https://goose-docs.ai/
8. OpenCode, https://opencode.ai/
9. Pi agent, https://pi.dev/

ma è difficile dare un elenco esaustivo perché è un terreno dove ogni giorno escono novità e prodotti con idee che migliorano i risultati.

Tutti questi sistemi possono essere estesi con la tecnica delle “skills”: risorse di tipo file, che descrivono come portare a termine un compito e che vengono attivati quando il sistema di harness ritiene di dover eseguire un passo dello sviluppo che richiede esattamente quelle attività.
In questo modo gli sviluppatori possono estendere le funzionalità del sistema di base per ottenere la copertura di svariate attività che non sono previste dal sistema così come esce dalla fabbrica.

Le skills non sono l’unico sistema di estensione delle capacità. Tipicamente tools e plugin sono disponibili per implementare attività come la lettura di file, l’esecuzione di comandi del sistema operativo o le ricerche in internet.

Alcuni di questi oggetti sono Open Source e quindi la possibilità di estendere il sistema con questo tipo di strumenti è portata al suo massimo livello.

Grazie alla possibilità di estensione sono nate collezioni di skills, comandi e tools aggiuntivi che prendono in carico lo svolgimento di molte attività comprese da uno sviluppo software.
La fantasia del “Vibe Coding” si è presto scontrata con la realtà dell'ingegneria del software: anche con l’aiuto dell’ Intelligenza Artificiale non si possono eludere le attività di raccolta dei casi d’uso, di stesura delle specifiche, di progettazione, di realizzazione del codice e quindi alla fine di verifica del lavoro svolto.

Costruire un prodotto è diverso dallo scrivere linee di codice.

Le collezioni di skill definiscono le operazioni da eseguire workflow per le attività da svolgere prima, attorno e dopo la scrittura del sorgente.
Abbiamo raccolte più generiche che implementano dei passi come l’intervista iniziale, la pianificazione, la progettazione, la scrittura ed il debug e altri sistemi più orientati alla raccolta formale di specifiche che poi vengono elaborate nei passi successivi.
Al primo gruppo appartengono, come esempi:

1. la raccolta "SuperPowers”, https://claude.com/it/plugins/superpowers
2. le “Agent-skills” di Addy Osmani, https://github.com/addyosmani/agent-skills
3. la raccolta “Grill me” di Matt Pocock, https://www.aihero.dev/grill-with-docs

I principali rappresentanti del secondo gruppo sono:

1. il metodo BMAD, https://github.com/bmad-code-org/bmad-method
2. Spec-kit, https://github.com/github/spec-kit
3. il sistema OpenSpec, https://github.com/Fission-AI/openspec
4. l’AI Unified Process, https://unifiedprocess.ai/index.html

Come Zucchetti, siamo interessati a verificare l’estendibilità dell’agente di harness OpenCode.
In particolare, vogliamo testare la possibilità di verificare la corrispondenza tra i casi d’uso, le specifiche e quindi il codice generato.

Se gli LLM sono in grado di generare a grande velocità linee di codice, non è detto che queste siano corrette sia dal punto di vista del programma in sé, ma anche della rispondenza del programma alle specifiche che hanno motivato la realizzazione.

Con gli LLM è nato un nuovo problema: il collo di bottiglia non è più la costruzione ma piuttosto la verifica. Abbiamo automatizzato la scrittura, non abbiamo ancora automatizzato il controllo di quanto viene scritto.

Per la validità del codice in sé abbiamo strumenti come compilatori, lint, test automatici, ma nessuno di questi ci può dire se il codice realizzato rispetta il desiderio che ha portato a scriverlo.



In questo capitolato chiediamo quindi di realizzare skill, eventualmente anche altri strumenti di estensione di OpenCode, che nel contesto di un progetto software possano verificare se la descrizione dei casi d'uso, raccolti nella fase di esplorazione con gli utenti, trova effettiva implementazione nei requisiti di progetto e se quei requisiti trovano effettiva implementazione nel codice finale del prodotto.
Le skills saranno integrate nello strumento OpenCode che verrà utilizzato per realizzare uno specifico progetto software esterno al capitolato. Il capitolato richiede di estendere lo strumento OpenCode che viene usato per governare il progetto software richiesto da un utente o da un imprenditore.

Stabiliti gli obiettivi di massima di un progetto, il primo passo è la stesura dei casi d'uso raccolti intervistando gli utenti che saranno i fruitori della procedura. Ci aspettiamo che questi casi d'uso siano stati processati verificandone la correttezza sia sintattica che semantica.
A questo punto possono intervenire le skills del capitolato per leggerne le caratteristiche e verificare che siano riportate correttamente nei requisiti. La stessa situazione, in fasi successive, avverrà tra i requisiti ed il codice prodotto.

Terminata la fase di esplorazione dei casi d'uso verranno realizzate le descrizioni dei requisiti, chiediamo di costruire un sistema coordinato di skills, o altri tools se necessario, per verificare se tutti i casi d'uso sono riportati correttamente nei requisiti.
Dati i requisiti e le specifiche di progetto verrà quindi realizzato il codice del programma, chiediamo anche in questo caso un sistema di skills, o altri tools se necessario, per verificare se il

comportamento del programma effettivamente implementa i casi d'uso e i requisiti espressi nelle fasi iniziali della realizzazione.

Il sistema richiesto deve occuparsi della verifica, non della realizzazione che resta affidata ai programmatori.

Le skill del capitolato sono estensioni dello strumento OpenCode per realizzare il progetto “esterno” al capitolato.

Caratteristiche e Requisiti Obbligatori

L’ azienda Zucchetti Spa chiede di sviluppare delle estensioni (skill, tools, comandi) al sistema di gestione del contesto per lo sviluppo del codice OpenCode che svolgano i seguenti compiti:

1. Lettura di file che rappresentano i diagrammi dei casi d’uso.
2. Lettura di documentazione aggiuntiva sui casi d’uso.
3. Lettura delle specifiche dei requisiti.
4. Confronto tra le informazioni ottenute dai casi d’uso e le specifiche dei requisiti.
5. Verifica della rispondenza delle specifiche dei requisiti con quanto indicato nei casi d’uso.
6. Lettura del codice sorgente e comprensione del comportamento del programma.
7. Confronto con le specifiche dei requisiti.
8. Verifica che il programma implementi effettivamente il comportamento richiesto dalle specifiche dei requisiti.
9. Realizzazione di una dashboard per riportare lo stato della realizzazione rispetto a quanto previsto dalle specifiche dei requisiti.

Il progetto richiede di verificare un insieme di casi d’uso, di specifiche dei requisiti e di codice. Questa verifica richiede di disporre materiale da controllare. Per questo consigliamo di utilizzare come base un proprio progetto preesistente o di basarsi su un esempio di progetto già svolto, autorevole per qualità, disponibile in rete.
Al termine del progetto didattico conseguente a questo capitolato,, quando saranno disponibili i suoi artefatti, sarà possibile verificare con esso anche il materiale creato durante lo svolgimento del progetto corrente, senza interferire con il suo avanzamento.

Vediamo in dettaglio i punti dell’elenco dei requisiti del progetto del capitolato.

Il primo punto richiede di definire come verranno specificati i casi d’uso. Poiché nascono tipicamente da diagrammi, andrà definito quale modello di scrittura dei diagrammi viene compreso.
Due formati facilmente gestibili dagli LLM sono “Mermaid”, che permette di scrivere diagrammi in markdown, e XMI che è il formato XML dei tool di disegno dei diagrammi UML.

Un'ulteriore possibilità è verificare se gli LLM riescano a leggere direttamente i diagrammi dai formati di immagine, come Png o jpeg.

Il secondo punto indica come i casi d'uso siano spesso corredati da documentazione che completa quando descritto dal diagramma.
Il sistema deve poter leggere le informazioni sui singoli casi d'uso anche da queste fonti, spesso testuali. Possiamo immaginare che siano in formato markdown.

Il terzo punto richiede di leggere le specifiche dei requisiti che sono nati dai casi d'uso. Tipicamente queste specifiche sono in formato testuale; possiamo immaginare che il formato preferenziale sia markdown.
È necessario comprendere cosa sia contenuto nei documenti per poter quindi procedere al confronto.

Il quarto punto porta i dati ricevuti dai precedenti punti al sistema di verifica, dove le specifiche dei requisiti e i casi d'uso devono essere confrontati.
Ogni caso d'uso deve trovare delle specifiche nei requisiti che determinano come verrà implementato nel codice.
Un caso d'uso potrebbe rientrare in più di una specifica di un requisito .

Il quinto punto prevede che i testi dei casi d'uso e delle specifiche devono essere valutati per verificare la rispondenza delle due descrizioni. Se non fosse possibile comprendere se effettivamente stiano cogliendo due aspetti dello stesso argomento, uno più descrittivo dall'esterno e uno più propositivo per la realizzazione, questa mancata coincidenza deve essere adeguatamente segnalata.

Sviluppato il programma, cioè scritto il codice corrispondete, arriviamo alle richieste del sesto punto: gli LLM sono capaci di comprendere il comportamento di un programma leggendone il codice. Chiediamo di approfondire questo aspetto per poter predisporre il prossimo punto.

Nel settimo punto chiediamo di confrontare il comportamento dedotto dall' LLM con le specifiche della procedura raccolte e sviluppate nella fase seguente alla stesura dei casi d'uso.

Arriviamo quindi all'operazione finale: verificare se il comportamento del programma, come codificato, sia conforme alle specifiche e realizzi effettivamente quanto desiderato, specificato nei requisiti, derivati dai casi d'uso.

Come ultimo punto richiediamo di sviluppare una dashboard per presentare i risultati di quanto è stato rilevato nelle attività precedenti.
Tale visualizzazione può essere anche "a caratteri" e non necessariamente grafica, visto che lo strumento OpenCode privilegia l'interfaccia a terminale.
Deve essere riportato lo stato di avanzamento dei lavori, evidenziando quali casi d'uso sono stati rilevati nelle specifiche e quali specifiche sono state rilevate nel codice.

La cosa importante è che ogni discrepanza tra quello che viene trovato nel codice e quello che indicavano le specifiche sia facilmente percepibile al programmatore.
Questo punto richiede precisione ed analisi dei file che fanno parte del progetto. Poiché queste attività non possono essere svolte in modo deterministico da un LLM si richiede espressamente che questa funzionalità sia realizzata sotto forma di “Plugin” per OpenCode o come applicativo esterno che viene richiamato da skill e comandi.
Come esempio di questa architettura suggeriamo di analizzare il sistema OpenSpec che è realizzato con comandi, skill e un eseguibile di appoggio.

Il progetto è molto esplorativo, se alcune parti sono consolidate, come ad esempio la scrittura di skill o comandi, altre parti si basano sulle capacità degli LLM che potrebbero non essere definite in modo certo.
L’azienda può mettere a disposizione l’accesso sia a modelli di grandi dimensioni, ma con un limite massimo di costo, sia a modelli di piccole dimensioni come Gemma 4 26B o Qwen 3.8 27B, senza limitazioni, per poter verificare la solidità delle realizzazioni anche con LLM ridotti.

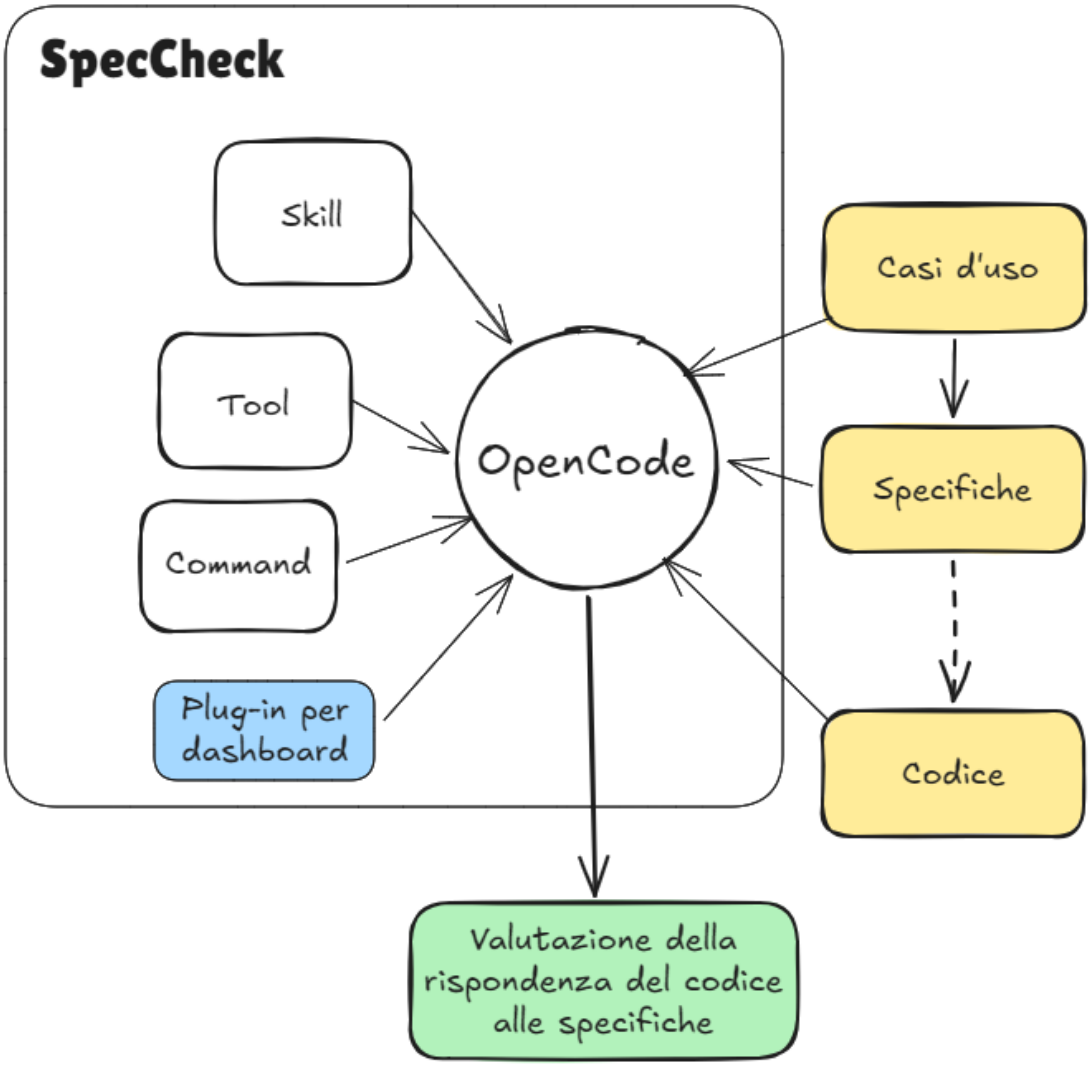
Il prodotto finale sarà un pacchetto di skill, tool ed almeno un plug-in per OpenCode. L’insieme di questi artefatti deve essere considerato un prodotto unico e a questo fine deve essere corredato di un sistema di installazione, che può essere un semplice script bash o powershell, per aggiungere in modo unitario l’insieme delle componenti di “SpechCheck” ad OpenCode.

Nella prima fase di questo progetto didattico, dove verrà sviluppato il PoC (Proof of concept) consigliamo di esplorare soprattutto gli algoritmi di confronto e la lettura dei diagrammi dei casi d’uso.

Nella seconda fase, viene sviluppato l’MVP (Minumum Viable Product) che deve realizzare i requisiti indicati come obbligatori, che di fatto è il sistema minimo per poter usufruire dei risultati attesi.

I requisiti opzionali portano ad un prodotto completo che può appunto essere sviluppato qualora i tempi a disposizione fossero compatibili con la realizzazione delle potenzialità indicate.

Per ogni fase, ma soprattutto a partire dall’MVP, è importante che alla realizzazione si accompagni una adeguata copertura con test automatici. Ci attendiamo infatti che il sistema sia soggetto a revisioni e quindi verificabile in modo adeguato solo attraverso la ripetizione automatizzata dei casi d’uso principali.



Schema di massima del progetto SpecCheck

Lo sviluppo per passaggi successivi, da zero a PoC, da PoC a MVP, richiede l'adozione di pattern architetturali di comprovata duttilità e robustezza. Ad ogni iterazione il codice dovrà essere revisionato, aumentato ed esteso. Quindi, sistemi fragili o senza una validità certa non sono idonei. Inoltre, lo spiccato carattere esplorativo del tema proposto aumenterà il numero delle revisioni necessarie proprio per le scoperte che verranno fatte sviluppando le prime interazioni.

Requisiti Opzionali

Il problema posto affronta la verificabilità di programmi creati ad alta velocità da sistemi di intelligenza artificiale.

I requisiti opzionali tendono a fornire un sistema utilizzabile come ambiente di verifica nel contesto di una pipeline di produzione del software.

Dashboard grafica: il sistema OpenCode è basato su una interfaccia TUI, testuale e ospitata in un terminale, che pur essendo efficace per tutte le attività proposte non è ottimale per la presentazione finale dei risultati trovati. Opzionalmente quindi chiediamo di sviluppare una interfaccia grafica, che può essere una pagina HTML, per presentare ed esplorare quanto rilevato come problematico nella realizzazione del software.

Verifica della copertura delle specifiche: il requisito obbligatorio richiede di confrontare le specifiche con il codice al fine di verificare se tutto quello che era stato specificato ha trovato la sua realizzazione.
Ma il codice potrebbe avere delle parti che non trovano corrispondenza, potrebbero cioè esserci delle linee di codice che implementano comportamenti al di fuori di quanto specificato.
Questo requisito chiede di sviluppare la capacità di identificare la copertura del codice dalle specifiche e quali parti realizzate non sono derivate da specifiche formali.

Lettura da file PDF: il sistema di raccolta dei casi d'uso e delle specifiche spesso porta alla realizzazione di file in formato PDF, in quanto questi sono facilmente leggibili da persone e realizzabili con molti strumenti di sviluppo e di diagrammazione.
Nei requisiti obbligatori accettiamo che i file dei diagrammi e le parti testuali dei casi d'uso e delle specifiche possano essere in formati facilmente esplorabili dagli LLM (es: martkdown), in questo requisito chiediamo di esplorare la possibilità di operare direttamente su file in formato PDF per leggere i casi d'uso e i loro diagrammi e le specifiche formali.

Suggerimenti

Lo strumento OpenCode:
https://opencode.ai/

Raccolte di skill da cui prendere spunto:
https://github.com/obra/superpowers
https://github.com/addyosmani/agent-skills
https://www.aihero.dev/grill-with-docs

Strumenti per la gestione delle specifiche con OpenCode da cui prendere spunto:



https://github.com/bmad-code-org
https://openspec.dev/
https://github.com/github/spec-kit

Strumento testuale per lo sviluppo di diagrammi UML:
https://plantuml.com

Altro strumento per lo sviluppo di diagrammi UML con testo esplorabile:
https://www.umlet.com/

Portale di raccolta di LLM e di sistemi correlati:
https://huggingface.co/

Parte di questo documento è stato revisionato da ChatGPT.

Variazioni ai requisiti

In corso d'opera non sarà possibile variare/modificare i requisiti minimi (obbligatori per accettare il prodotto). Sarà invece possibile variare i requisiti opzionali, in quanto saranno i gruppi vincitori dell'appalto a modificarli / eliminarli / aggiungerli.

Documentazione

Il progetto dovrà essere supportato dalla documentazione minima richiesta per il corso di Ingegneria del software e dovrà essere fornito un manuale per l'utilizzo ed un manuale per chiunque voglia estendere le skill, i comandi ed i tool.

Garanzia e Manutenzione

L'azienda Zucchetti SPA è interessata a questo progetto come dimostrazione della fattibilità dell'obiettivo utilizzando le tecnologie degli LLM e dei gestori di harness. Costituirà titolo preferenziale nella valutazione delle proposte la pubblicazione del progetto sul sito “github.com” o altri repository pubblici, in conformità con i relativi requisiti di natura open-source, per favorire la continuità del prodotto risultante.

Rinvio

Per tutto quanto non previsto nel presente capitolato, sono applicabili le disposizioni contenute nelle leggi e nei collegati per la gestione degli appalti pubblici.



Contatti

Per qualsiasi informazione la persona di riferimento in Zucchetti è Gregorio Piccoli all'indirizzo email gregorio.piccoli@zucchetti.it